NutriBot — Documentação do Código

Diário alimentar com IA. O utilizador descreve o que comeu em linguagem natural, a IA analisa as macros, dá feedback honesto (com "humor" visual) e guarda tudo num diário do dia.

Stack

- **Backend**: Node.js + Express (ESM)
- **Base de dados**: SQLite (ficheiro local `src/nutribot.db`)
- **IA**: Groq SDK com o modelo `llama-3.3-70b-versatile`
- **Frontend**: HTML + CSS + JS vanilla (sem framework), com Server-Sent Events (SSE) para streaming
- **Extras**: Web Speech API para entrada por voz

Estrutura de ficheiros

```
projeto m7 nutribot/
├── package.json              # Dependências e script "start"
├── src/
│   ├── api.js                # Servidor Express + rotas
│   ├── db.js                 # Camada SQLite (tabelas e queries)
│   ├── groqClient.js         # Cliente Groq + system prompt + tools
│   ├── nutriParser.js        # Extrai macros de texto livre via IA
│   └── nutribot.db           # Base de dados SQLite (gerada automaticamente)
└── Public/
    ├── index.html            # UI + lógica do frontend (tudo num ficheiro)
    └── nutribot.css          # Estilos (incluindo temas de humor)
```

Fluxo geral

1. O utilizador abre o site → modal pede **nome, idade, peso, altura, objetivo**.
2. Os dados são guardados em SQLite e o `user.id` fica em `localStorage`.
3. O utilizador escreve algo no chat (ex: *"comi 2 ovos e morangos"*).
4. O frontend envia a mensagem para `/chat` (SSE).
5. O backend chama a IA (Groq) com o histórico recente e o perfil do utilizador.
6. A IA pode:
   - **Responder com texto** → análise nutricional com `MOOD:` na primeira linha.
   - **Chamar uma tool** → eliminar refeições do diário.
7. Se o texto parece descrever comida (`looksLikeFood`), o frontend chama também `/nutrition/parse` que extrai macros via IA e grava no diário.

Ficheiros em detalhe

src/api.js — Servidor e rotas

Ponto de entrada (`npm start` → `node src/api.js`). Cria o app Express, serve `Public/` como ficheiros estáticos e define as rotas REST + SSE.

Funções e rotas:

- **limparMarkdown(texto)** — remove `**negrito**` , `*itálico*` e `# headings` antes de enviar a resposta ao frontend (a IA por vezes ignora a instrução de não usar markdown).
- **POST /users** — cria utilizador. Valida que `nome, idade, peso, altura, objetivo` existem, chama `saveUser` e devolve o user criado.
- **GET /users/:id** — devolve o utilizador (usado quando o frontend tem `user_id` no `localStorage`).
- **GET /chat/history** — devolve as últimas 5 mensagens do utilizador, para repopular o chat ao recarregar a página.
- **executeTool(toolName, args, userId)** — switch que executa as tools chamadas pela IA:
  - `delete_food_entry` — procura por nome (match parcial em ambos os sentidos) e apaga.
  - `delete_last_food_entry` — apaga a entrada mais recente.
  - `delete_all_food_entries` — limpa todo o diário do utilizador.
- **GET /chat** — **rota principal**, usa **Server-Sent Events**.
  1. Carrega o user e o histórico das últimas 5 mensagens.
  2. Chama `chatWithTools` (Groq).
  3. Se a resposta for `tool_call` , executa a tool e envia ao cliente:
     - `event: tool_action` → frontend atualiza o DOM (remove items do diário).
     - `event: done` → fecha o stream.
  4. Se for texto normal:
     - Procura `MOOD:happy|ok|stressed|angry` na primeira linha.
     - Envia o mood em `event: mood` e o resto do texto em `data:` .
  5. Guarda o par mensagem/resposta em `chat_history` .
- **POST /nutrition/parse** — recebe texto livre, chama `parseNutritionFromText` (extrai JSON de macros via IA), grava em `food_diary` e devolve a entrada.
- **GET /nutrition/diary** — lista todas as entradas do diário do utilizador.
- **DELETE /nutrition/diary/:id** — apaga uma entrada pelo botão ✕ na UI.

src/db.js — Base de dados

Abre o SQLite em modo WAL (Write-Ahead Logging) para permitir leituras/escritas concorrentes sem bloqueio. Cria três tabelas se ainda não existirem.

Tabelas:

Tabela	Colunas
`users`	id, nome, idade, peso, altura, objetivo, created_at
`food_diary`	id, user_id, alimento, kcal, proteina, carboidratos, gordura, created_at
`chat_history`	id, user_id, user_message, ai_response, created_at

Helpers internos (envolvem `db.run/all/get` em Promises):

- `run(sql, params)` → devolve `{ lastID, changes }` (INSERT/UPDATE/DELETE).
- `all(sql, params)` → devolve array de linhas.
- `get(sql, params)` → devolve uma única linha.

Funções exportadas:

- `saveUser(nome, idade, peso, altura, objetivo)` — INSERT em `users`.
- `getUser(id)` — SELECT por id.
- `saveFoodEntry(userId, alimento, kcal, proteina, carboidratos, gordura)` — INSERT em `food_diary`.
- `getAllFoodEntries(userId)` — lista refeições ordenadas por id DESC (mais recentes primeiro).
- `deleteFoodEntry(id)` — apaga uma refeição.
- `deleteAllFoodEntries(userId)` — apaga todas as refeições do utilizador.
- `getLastFoodEntry(userId)` — última refeição registada.
- `saveChatMessage(userId, userMessage, aiResponse)` — INSERT em `chat_history`.
- `getRecentChatHistory(userId, limit=5)` — últimas N mensagens, **invertidas** no fim para ficarem por ordem cronológica.

---

src/groqClient.js — Cliente da IA

Apesar do nome, usa **Groq** (não Gemini). Define as tools, o system prompt e duas funções de chamada.

Tools registadas (function calling):

- `delete_food_entry(nome)` — eliminar uma refeição pelo nome.
- `delete_last_food_entry()` — eliminar a última.
- `delete_all_food_entries()` — eliminar todas as de hoje.

As descrições das tools são **muito explícitas** a dizer que só devem ser chamadas quando o utilizador usa "elimina/apaga/remove/cancela" e NUNCA quando descreve comida ("comi/almocei/jantei"). Isto é uma defesa contra falsos positivos do modelo.

createSystemPrompt(user) — gera o prompt principal:

- Se não houver user, pede os dados.
- Se houver user, calcula:
  - **TMB** (Mifflin-St Jeor) — taxa metabólica basal.
  - **TDEE** = TMB × fator de atividade (`sedentario` 1.2, `leve` 1.375, `moderado` 1.55, `intenso` 1.725, `atleta` 1.9).
  - **Meta calórica**: TDEE −500 (perder), +300 (ganhar) ou TDEE (manutenção).
  - **Proteína**: 1.8 g/kg (perder), 2.0 g/kg (ganhar), 1.6 g/kg (manutenção).
  - **Gordura**: 25% das calorias.
  - **Hidratos**: o resto.
  - **Meta por refeição** = total ÷ nº de refeições/dia (default 5).
- Inclui regras estritas sobre o formato da resposta: linha `MOOD:`, lista de alimentos, totais, ✅/⚠️/❌, julgamento direto e sugestão concreta.

chatWithTools(userMessage, history, user) — chamada principal:

- Constrói o array de `messages` (system + histórico expandido em pares user/assistant + mensagem atual).
- Envia para Groq com `tools`, `tool_choice: 'auto'`, `temperature: 0.3`, `max_tokens: 600`.
- Se o modelo decidir chamar uma tool, devolve `{ type: 'tool_call', tool_calls: [...] }`.
- Caso contrário, devolve `{ type: 'text', text: ... }`.

generateJson(prompt) — chamada com `response_format: json_object` e `temperature: 0` para extração estruturada (usada pelo nutriParser).

---

src/nutriParser.js — Extração de macros

Uma única função, parseNutritionFromText(text):

1. Constrói um prompt a pedir um JSON estrito com `alimento, kcal, proteina, carboidratos, gordura`.
2. Chama `generateJson` (que força `json_object`).
3. Remove possíveis ```` ```json ```` ou ```` ``` ```` à volta.
4. `JSON.parse` e devolve o objeto.

Resultado: a partir de *"comi 2 ovos mexidos"* → `{ alimento: "Ovos mexidos", kcal: 200, proteina: "14g", ... }`.

Public/index.html — Frontend completo

Inclui HTML, CSS link e ~400 linhas de JS inline.

Inicialização (linhas 109-125):

- Lê `nutribot_user_id` do `localStorage`.
- Se existir, faz `GET /users/:id`, esconde o modal e carrega histórico + diário.

Modal de cadastro (linhas 127-158):

- Valida campos, faz `POST /users`, guarda `user.id` no `localStorage`.

applyMood(mood) — muda a classe do `<body>` para `theme-happy/ok/stressed/angry` (CSS troca as cores) e altera a label do header:

- happy → "Estás lá! 💪"
- ok → "Não está mauzito! 👌"
- stressed → "Epa tu me digas mais nada... 😮 👊"
- angry → "Desisto de ti. 🤦‍♀️"

Speech-to-Text (linhas 184-213):

- Usa `webkitSpeechRecognition` em `pt-PT`.
- Se o browser não suportar, esconde o botão do microfone.

sendMessage() — o núcleo do chat:

1. Mostra a mensagem do utilizador.
2. Cria a "bolha" do bot com um cursor a piscar.
3. Faz `fetch('/chat?message=...&user_id=...')` e lê o ReadableStream.
4. Faz parse linha a linha do SSE:
   - `event: mood` → chama `applyMood`.
   - `event: tool_action` → remove items do DOM (delete_one ou delete_all).
   - `data: "[DONE]"` → fim.
   - `data: "<texto>"` → concatena ao `fullResponse` e atualiza a bolha.
5. No fim, se `looksLikeFood(text)` for verdade e não foi uma tool call, chama `parseAndSaveFood` para gravar no diário.

looksLikeFood(text) — heurística simples: procura "comi/almocei/jantei/bebi/refeição/lanche..." no texto.

parseAndSaveFood(text) — `POST /nutrition/parse` → adiciona ao DOM via `addDiaryItem`.

addDiaryItem(entry) — cria o card no diário com nome, kcal, macros (P/C/G) e botão ✕ para apagar. Faz `prepend` (entradas novas ficam no topo) e chama `refreshTotal`.

refreshTotal() — soma todas as kcal visíveis e atualiza o badge "X kcal" no header do diário.

deleteEntry(id, itemEl) — `DELETE /nutrition/diary/:id` e remove o card do DOM.

Variáveis de ambiente

Cria um ficheiro `.env` na raiz:

```
GROQ_API_KEY=gsk_...
PORT=3000        # opcional, default 3000
```

Como correr

```
npm install
npm start
```

Abre `http://localhost:3000`.

Pontos a saber / pegadinhas

- A dependência `@google/genai` ainda está no `package.json` mas não é importada — provavelmente o projeto começou com Gemini e foi migrado para Groq. Podes remover com `npm uninstall @google/genai`.
- O frontend chama `/nutrition/parse` **em paralelo** com `/chat` sempre que o texto "parece comida" — são dois pedidos à IA por refeição registada.
- A BD não tem `FOREIGN KEY` nem índices em `user_id` — não é problema na escala atual mas seria a primeira otimização se o projeto crescesse.
- `getAllFoodEntries` devolve TODO o histórico do utilizador, não apenas o dia atual — o nome "Diário de Hoje" na UI é um pouco enganador.
- O parsing dos SSE no frontend assume que cada `read()` traz linhas inteiras. Em redes lentas pode partir uma linha ao meio — funciona na maioria dos casos mas não é 100% robusto.